

PROJECT DOCUMENTATION

EventForce Management System

Salesforce Implementation

A Salesforce CRM solution that digitizes and automates event management operations for
Event Planner Solutions

Team lead: R. Helan Bobby

Team member 1: J.V. Pragalya

Team member 2: M. Abinaya

Table of Contents

1. Project Overview	1
2. Objectives	1
3. Phase 1: Requirement Analysis and Planning	1
4. Phase 2: Backend Development and Configuration	2
5. Phase 3: UI/UX Development and Customization	7
6. Phase 4: Data Migration, Testing and Security	9
7. Phase 5: Deployment, Documentation and Maintenance	11
8. Conclusion	11

1. Project Overview

The **EventForce Management System** is a Salesforce-based Customer Relationship Management (CRM) platform built for **Event Planner Solutions**. It helps event planners manage client bookings, vendor coordination, venue reservations and feedback collection within a single platform. The system automates key processes such as event scheduling, cancellation approvals and client reminders, and gives real-time visibility into operations through reports and dashboards.

The project was developed in five phases: requirement analysis and planning, backend development and configuration, UI/UX development and customization, data migration, testing and security, and finally deployment, documentation and maintenance. All components were built and tested in a Salesforce Developer Edition org. The solution consists of six custom objects, a record-triggered flow, an approval process, a validation rule, Apex classes and triggers, a Lightning app with a report and dashboard, and a role-based security model.

2. Objectives

The primary objective is to create a comprehensive CRM platform that digitizes and automates event management operations for Event Planner Solutions. The main goals of the project are:

- Streamline client bookings, venue reservations, vendor coordination and feedback collection.
- Centralize data and improve collaboration among event coordinators, vendor managers and clients.
- Automate workflows such as reminders, cancellation approvals and venue availability updates.
- Ensure accurate data entry and event budget calculation through validation rules and formula fields.
- Protect data through role-based access control.
- Provide data-driven insights through integrated reports and dashboards.

Targeted business value: reduce manual processes by 60%, improve booking efficiency by 40%, enhance client satisfaction through timely communication, and support business growth with data-driven insights.

3. Phase 1: Requirement Analysis and Planning

3.1 Functional Scope

- Custom objects: Event, Client, Vendor, Venue, Feedback and Event Vendor (junction object).

- Relationship modeling using Lookup, Master-Detail and Many-to-Many (Event and Vendor) relationships.
- Flow automation for reminders, feedback notifications and approval updates.
- Validation rules and formula fields for accurate data entry and event budget calculation.
- Approval process for event cancellation requests with email alerts to coordinators and clients.
- Profiles, roles and permission sets to manage user-level access and security.
- Reports and dashboards for event summary, vendor performance and upcoming event insights.

3.2 Stakeholder Mapping

Stakeholder	Responsibility
Clients	Request event services, provide feedback and view event updates.
Event Coordinators	Manage event scheduling, client communication and approval workflows.
Vendors	Handle assigned event services such as catering, decor and logistics.
Venue Managers	Manage venue availability and reservations.
Event Administrators	Oversee system operations, security and data monitoring.

3.3 Execution Roadmap

Step	Activity	Description
1	Object and Field Setup	Create the Event, Client, Vendor, Venue, Feedback and Event Vendor objects.
2	Automation Flows	Implement record-triggered flows for event reminders and approvals.
3	Validation Rules and Formula Fields	Enforce data accuracy and budget calculations.
4	Profiles, Roles and Permission Sets	Configure user access and record visibility.
5	Approval Process	Automate the event cancellation workflow with notifications.
6	Apex Classes and Triggers	Update venue availability and prevent double bookings.
7	Reports and Dashboards	Create analytics for upcoming events and performance insights.

4. Phase 2: Backend Development and Configuration

This phase built the core functional engine of the system using Salesforce Flows, Apex logic and security configuration. Declarative tools (record-triggered flows, validation rules, formula fields and approval processes) automate booking confirmations, reminders and cancellation approvals. Apex classes and triggers handle advanced logic such as venue availability updates and prevention of double bookings. The work is organized into milestones as described below.

4.1 Salesforce Developer Account (Milestone 1)

Salesforce is a customer success platform that helps businesses sell, service, market, analyze and connect with customers, and stores data securely in the cloud. A developer org was created and activated as follows:

- Open `developer.salesforce.com/signup` and complete the sign-up form (name, email, role as Developer, company, country, postal code and a unique username in email format).
- Open the verification email, click *Verify Account*, set a password and answer the security question.
- After this, the user is redirected to the Salesforce Setup page.

4.2 Data Architecture: Objects, Tabs, Fields and Relationships (Milestones 2 to 4)

A Salesforce object is a database table that stores organization-specific data as fields (columns) and records (rows). The following custom objects were created from Setup > Object Manager > Create > Custom Object. Each object has *Allow Reports* and *Allow Search* enabled and the *Notes and Attachments* related list added to its page layout. The Event object additionally has *Allow Activities* and *Track Field History* enabled, and the object is set to Deployed.

Object	Record Name	Fields
Event__c	Event Name (Text)	Event Name, Event Date, Event Type, Event Status, Event Budget
Client__c	Client Name (Text)	Client Name, Email, Phone, Address, Country, City
Vendor__c	Vendor Name (Text)	Vendor Name, Email, Phone, Service Type, Status
Venue__c	Venue Name (Text)	Venue Name, Address, Location (URL), Capacity, Availability Status
Feedback__c	Feedback ID (Auto Number, format F-{0000}, starting at 0)	Feedback ID, Rating, Comments
EventVendor__c	Junction object	Links Event and Vendor (many-to-many)

A custom tab was created for every object (Setup > Tabs > Custom Object Tabs > New) so that users can view each object's data at a glance. Fields were added from Object Manager > Fields & Relationships > New, choosing the data type carefully for each field.

The relationships between the objects are summarized in the ER diagram below. Event is linked to Client and Venue by lookups; Feedback is linked to Event and Client by lookups; and EventVendor__c joins Event and Vendor through two Master-Detail relationships.

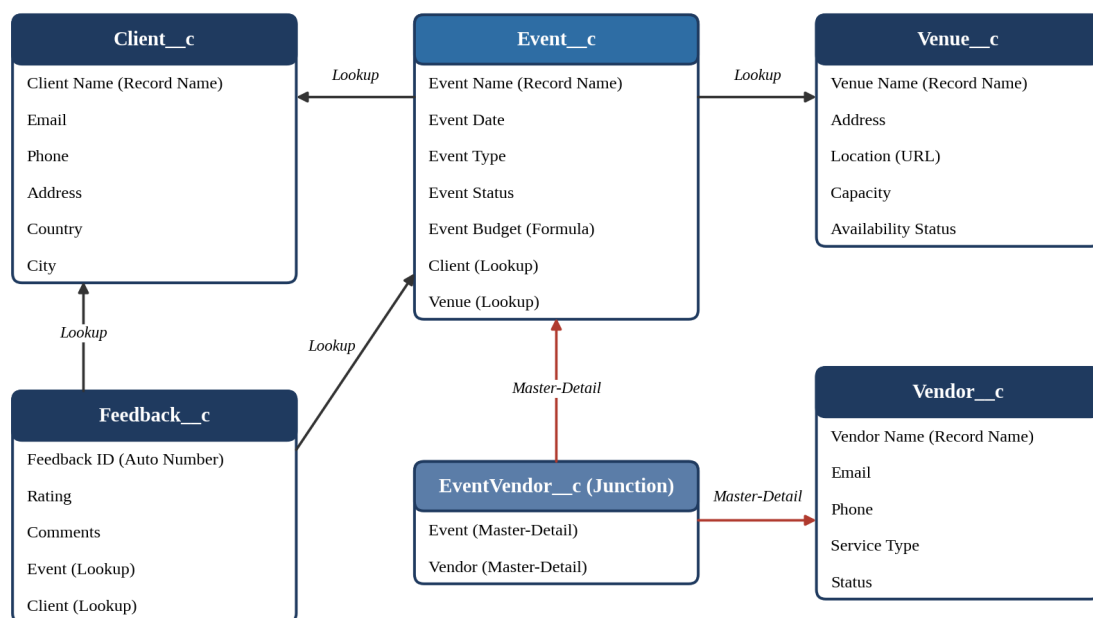


Figure 1: Entity Relationship (ER) diagram of the EventForce data model. Event and Vendor have a many-to-many relationship implemented through the EventVendor__c junction object.

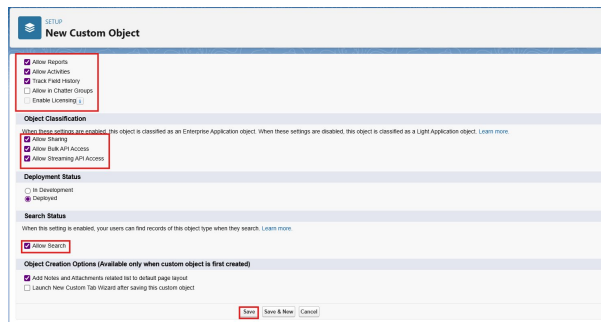


Figure 2: Creation of the Event custom object

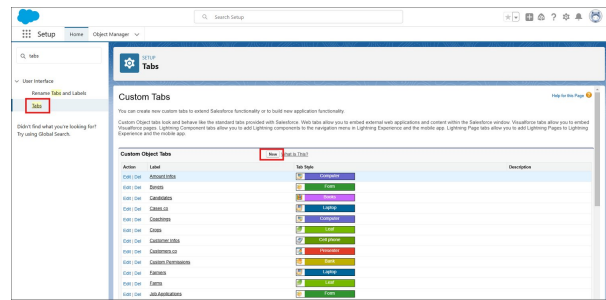


Figure 3: Custom object tabs created for the application

Field Configuration

Object	Field	Data Type	Values / Details
Event	Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event	Event Date	Date	Date of the event
Event	Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event	Event Budget	Formula (Currency)	Calculated from Event Type (see below)
Client	Email, Phone, Address	Email, Phone, Text Area	Contact details of the client
Client	Country	Picklist	India, Philippines, Saudi Arabia, USA, Switzerland, Thailand, Maldives, Dubai, Sri Lanka, Bangkok, Singapore, Myanmar, France, Nepal
Client	City	Picklist	Hyderabad, Mumbai, Delhi, Bangalore, Chennai, Alaminos, Angeles City, Palayan, Quezon City, San Juan, Riyadh, Jeddah, Tayma, Dammam, Medina, New York, Los Angeles, Chicago, Houston, Columbus, Geneva, Basel, Lausanne, Bern, Lucerne
Vendor	Service Type	Multi-Select Picklist	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor	Email, Phone	Email, Phone	Contact details of the vendor
Vendor	Status	Picklist	Available, Booked, Cancelled
Venue	Address, Location, Capacity	Text Area, URL, Number	Venue address, map link and seating capacity
Venue	Availability Status	Picklist	Available, Reserved, Not Available
Feedback	Rating	Picklist	1, 2, 3, 4, 5
Feedback	Comments	Long Text Area	Free-text comments from the client

Event Budget formula field. The Event Budget is a currency formula field that assigns a budget based on the selected Event Type:

Event Type	Wedding	Corporate	Birthday	Anniversary	Festival	Concert	Other
Event Budget	50,000	30,000	10,000	20,000	60,000	40,000	15,000

```

CASE(TEXT(Event_Type__c),
  "Wedding", 50000, "Corporate", 30000, "Birthday", 10000, "Anniversary", 20000,
  "Festival", 60000, "Concert", 40000, "Other", 15000, 0)

```

Relationship Setup and Lookup Filter

- **Event:** lookup fields to *Venue* and *Client*.
- **Feedback:** lookup fields to *Event* and *Client*.
- **Many-to-many (Event and Vendor):** Salesforce does not allow a direct many-to-many relationship, so the *Event Vendor* junction object (EventVendor__c) was created with two Master-Detail fields, one to Event and one to Vendor.
- **Lookup filter:** on the Feedback object's Event lookup, a filter was enabled so that after a Client is selected only that client's events appear. The filter criteria is *Event: Client equals Feedback: Client*.

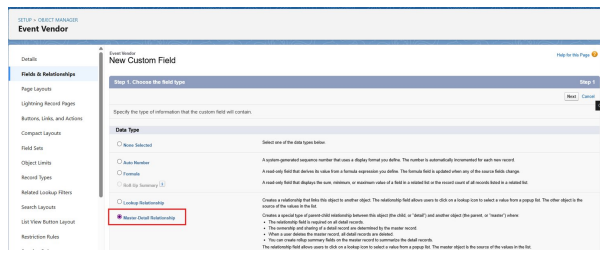


Figure 4: Master-Detail relationships on the Event Vendor junction object

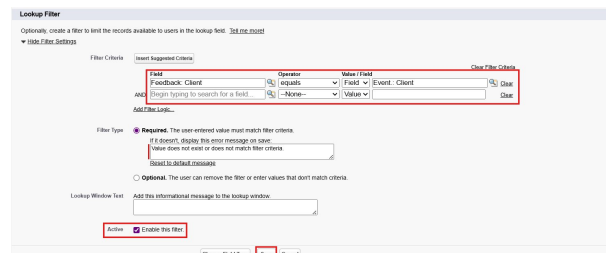


Figure 5: Lookup filter on the Feedback Event field

4.3 Validation Rule (Milestone 5)

Validation rules check whether data meets the specified criteria when a user saves a record. If the criteria are not met, an error message is shown and the record is not saved. A rule was created on the Client object (Object Manager > Client > Validation Rules > New):

Property	Value
Rule name / Status	Email_Valid_Address / Active
Error condition formula	NOT(REGEX(Email__c, "[a-zA-Z0-9._]+@[a-zA-Z0-9._]+\.[a-zA-Z]{2,}\$"))
Error message	Please Enter Valid Email Address
Error location	Field: Email

4.4 Approval Process for Event Cancellation (Milestone 6)

The approval process manages event cancellation requests. It must be implemented after the Profiles, Roles and Users milestones (Section 6) are complete. First, four classic email templates were created (Setup > Classic Email Templates):

Email Template	Purpose
Event Cancellation Request Notification	Sent to the approval team when a cancellation request is submitted.
Email to Client After Cancellation	Informs the client that the event has been canceled, with event name and date.
Approval Notification to Event Owner	Tells the event owner that the cancellation was approved and the status is Canceled.
Rejection Notification to Event Owner	Tells the event owner that the cancellation request was rejected.

The approval process **Event_Cancellation_Process** was then created on the Event object with the Standard Setup Wizard:

Setting	Configuration
Entry criteria	Event Status equals Pending Cancellation
Approver	Automatically assigned to a specific user or the manager of the submitter

Setting	Configuration
Approval email template	Event Cancellation Request Notification
Approval page fields	Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type, Venue
Initial submitters	Event Owner
Initial submission action	Email alert to the Event Coordinator role
Final approval actions	Field update: Event Status = Canceled; email alert to the Event Owner
Final rejection actions	Email alert to the Event Owner; field update: Event Status = Rejected
Approval step	Manager Review (all records enter this step; approver is a specific/related user). The process is activated from its detail page.

Figure 6: Entry criteria of the approval process

Figure 7: Approval process detail page

4.5 Flow: 3-Day Client Reminder (Milestone 7)

Flows are visual automation tools that collect and update data and guide users through business processes without code. **Scenario:** when an Event is created or updated with Status = Confirmed and a client with a valid email address, the system sends a reminder email to the client 3 days before the Event Date.

Flow Element	Configuration
Flow type / Object	Record-Triggered Flow on Event; runs when a record is created or updated
Entry condition	Event Status equals Confirmed; optimized for Actions and Related Records
Scheduled path	Path 3_Days_Before; time source Event Date; offset 3 days before
Text templates	Subject: “Reminder: Your Event {!\$Record.Name} is in 3 days”. Body: greets the client by name and lists the event name, date, venue and type, ending with “We look forward to seeing you! - EventForce Team”
Action	Send Email (Alert_Client_3Day_Reminder): recipient is the Client email, CC is the Event Owner; subject and body use the text templates
Save	Flow label “Client Reminder - 3 Days Before”; saved and activated

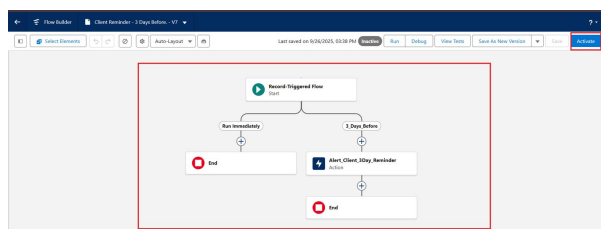


Figure 8: Completed Client Reminder flow with scheduled path

Figure 9: Send Email action configuration

4.6 Apex Classes and Triggers (Milestones 8 to 10)

Component	Type	Purpose
VenueStatusHelper	Apex class	Sets the Venue Availability Status to Reserved when an event is Confirmed and back to Available when the event is Canceled.
EventTrigger13	Trigger (after insert, after update on Event__c)	Calls VenueStatusHelper to keep venue availability current.
PreventDoubleBooking	Trigger (before insert, before update on Event__c)	Blocks two events at the same venue on the same date with the error “This Venue is already booked on this date.”
BatchCompleteEvents	Batch Apex (Database.Batchable)	Marks all events dated before today as Completed.
ScheduleCompleteEvents	Schedulable Apex	Runs the batch job with a batch size of 200; scheduled as “Daily Event Completion” at 8:00 PM on all days.

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}

trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}
```

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){ if(ev.Venue__c != null) venueIds.add(ev.Venue__c); }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){ venueEventMap.put(e.Venue__c, new List<Event__c>()); }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');
```

5. Phase 3: UI/UX Development and Customization

This phase created a simple, user-friendly interface in Salesforce Lightning. A custom Lightning app brings all modules (Events, Clients, Vendors, Venues and Feedback) into a single workspace, while reports and dashboards visualize event schedules, vendor performance and client feedback.

5.1 Lightning App (Milestone 11)

- Setup > App Manager > New Lightning App; the app name is **Event Planner** and a logo image was uploaded.
- App options and utility items were left at their defaults.
- Navigation items selected: Event, Client, Vendor, Venue, Feedback, Reports and Dashboards.

- The app was assigned to the System Administrator profile and verified through the App Launcher.

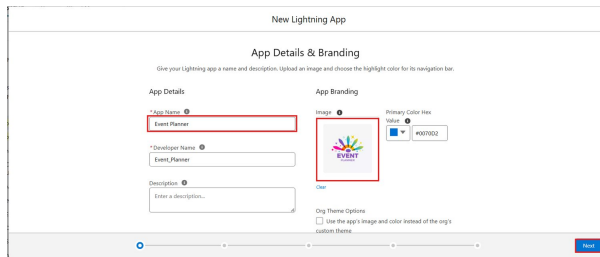


Figure 10: Event Planner app details and branding

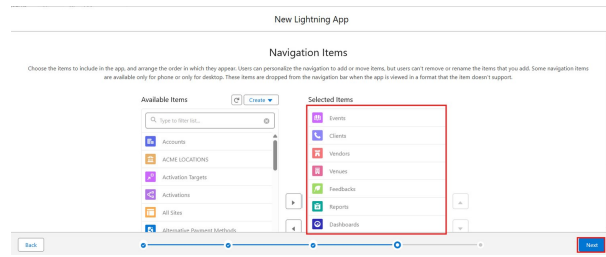


Figure 11: Navigation items of the Event Planner app

5.2 Reports (Milestone 12)

A report folder named **EventForce Report** was created and shared with the Event Coordinator and Vendor Manager roles with View access. At least 10 records were created in every object, with all fields filled, before building the report.

Report Setting	Configuration
Report name	Upcoming Events by Month
Report type	Events with Client and Venue
Columns	Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget
Filters	Show Me = All Events; Event Date on or after Today (or Next 30 Days); Event Status not equal to Canceled
Grouping	Event Date grouped by Calendar Month
Summaries	Row Count and SUM of Event Budget (total budget per month)
Chart	Column or Line chart with Event Date (Calendar Month) on the X-axis

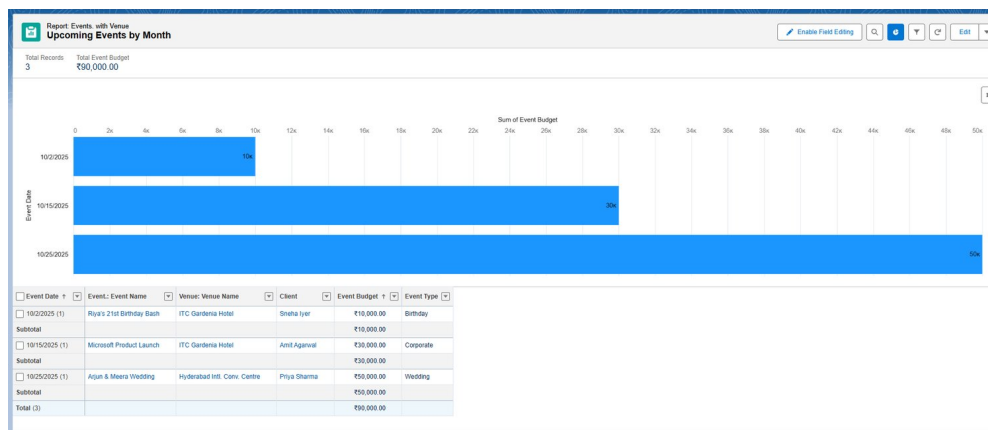


Figure 12: Upcoming Events by Month report

5.3 Dashboard (Milestone 13)

The **EventForce Operations Dashboard** was created from the Dashboards tab. A widget was added from the Upcoming Events by Month report as a donut chart showing a maximum of 6 values, with the subtitle “Upcoming Events Overview” and the Light widget theme.

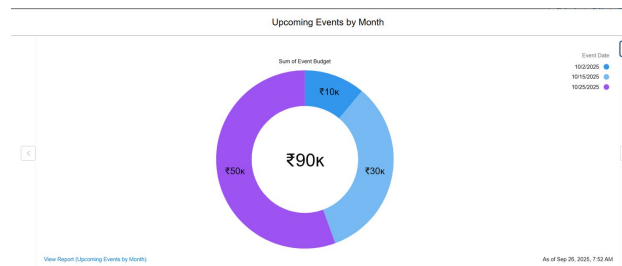


Figure 13: EventForce Operations Dashboard – upcoming events donut chart

6. Phase 4: Data Migration, Testing and Security

This phase prepared and secured the system before deployment. Records for Events, Clients, Vendors, Venues and Feedback were imported with the Data Import Wizard after pre-validation checks to avoid duplicates and maintain consistency. User accounts were created for the Event Admin, Event Coordinator, Vendor Manager and Client, each with a profile and role matching their access level.

6.1 Profiles (Milestone 14)

A profile is a collection of settings and permissions that defines what a user can do in Salesforce. A profile can be assigned to many users, but each user has only one profile. Four custom profiles were created:

Profile	Cloned From	Object Permissions
Event Admin	System Administrator	Full access (inherits Modify All Data, Customize Application and full CRUD on all objects)
Event Coordinator	Standard Platform User	Event: Create, Read, Edit, Delete; Client: Read, Edit; Vendor: Read; Venue: Read; Feedback: Create, Read
Vendor Manager	Standard Platform User	Vendor: Create, Read, Edit, Delete; Event, Venue, Client and Feedback: Read only
Client Profile	Standard Platform User	Event: Read only (own events); Feedback: Create, Read; Vendor and Venue: no access

For the Event Coordinator, Vendor Manager and Client profiles, Event Planner was set as the default custom app, the session timeout was set to 2 hours of inactivity, passwords were set to never expire, and the minimum password length was set to 8.

6.2 Roles and Users (Milestones 15 and 16)

Roles define the reporting hierarchy and data access. **Event Admin** was created under the CEO role, and **Event Coordinator**, **Vendor Manager** and **Client** were created below Event Admin. A user was then created for each role and profile. The Event Admin user uses the Salesforce license, and the other users use the Salesforce Platform license.

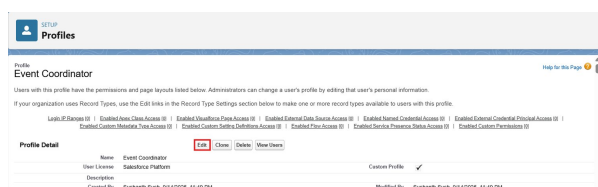


Figure 14: Event Coordinator profile detail

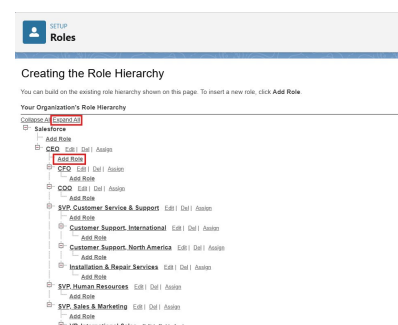


Figure 15: Role hierarchy with the Event Admin role

6.3 Permission Set (Milestone 17)

A permission set grants additional access without changing a user's profile. The **Feedback Manager** permission set was created with Read, Create, Edit and Delete permissions on the Feedback object and assigned to the Event Coordinator role user.

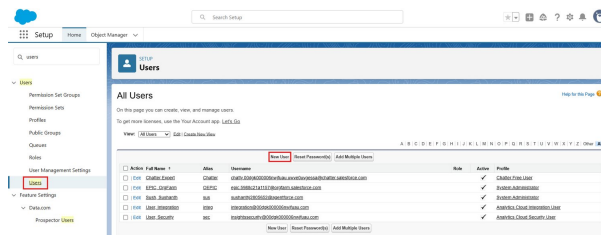


Figure 16: Users created for each role

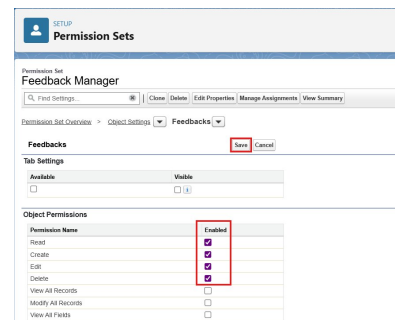


Figure 17: Feedback Manager permission set

6.4 Sharing Settings (Milestone 18)

Organization-Wide Defaults (OWD) set the baseline access to records, which can then be extended by role hierarchy, sharing rules, manual sharing and Apex sharing. For the Event object, both Default Internal Access and Default External Access were set to **Private**. To give vendors controlled visibility, the sharing rule **Event_Sharing_For_Vendors** was created: records owned by members of the *Event Coordinator* role are shared with the *Vendor Manager* role with *Read Only* access.

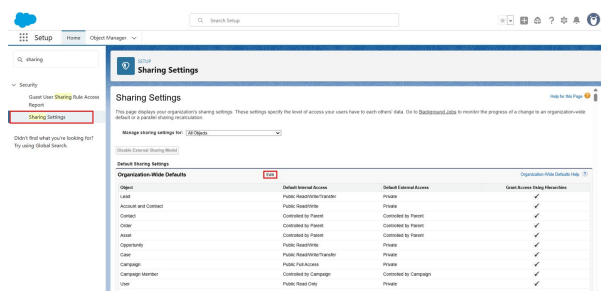


Figure 18: Organization-Wide Defaults for the Event object

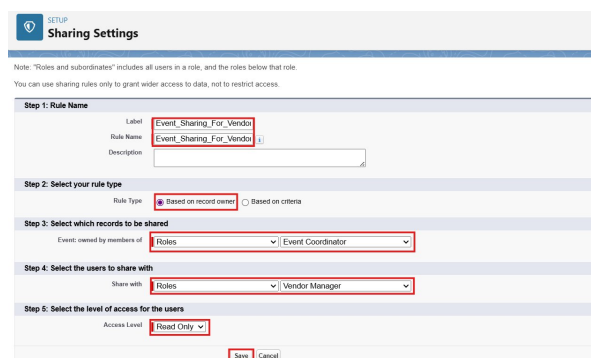


Figure 19: Event_Sharing_For_Vendors sharing rule

6.5 Testing and Validation Coverage

Testing validated data accuracy, automation performance and user permissions. The table summarizes the features covered and the behavior each is designed to deliver.

Area	Feature Tested	Expected Behavior
Data accuracy	Email validation rule	An invalid client email address is rejected with the message "Please Enter Valid Email Address".
Data accuracy	Event Budget formula; lookup filter	Budget follows the Event Type; the Feedback Event lookup lists only the selected client's events.
Data accuracy	Data import	Records are loaded after pre-validation to avoid duplicates.
Automation	Approval process	Status moves to Canceled on approval or Rejected on rejection, and the owner is notified by email.
Automation	Client reminder flow	Confirmed events trigger a reminder email to the client 3 days before the event date.
Automation	Apex triggers and batch job	Venue status is updated, double bookings are blocked and past events are marked Completed nightly.

Area	Feature Tested	Expected Behavior
Permissions	Profiles, permission set, OWD and sharing rule	Each user sees and edits only what their profile, role, permission set and sharing rules allow.

7. Phase 5: Deployment, Documentation and Maintenance

7.1 Deployment Strategy

- All configurations, customizations and automations were built and tested in a Salesforce Developer Edition org.
- The aim is to understand how a real-world Salesforce CRM solution is prepared for deployment after configuration, testing and security setup.
- Production deployment was not performed because Developer Edition orgs are standalone and not connected to production instances.
- Real deployments use sandbox environments, change sets or Salesforce DevOps tools (such as GitHub CI/CD pipelines or Salesforce DX).
- This phase therefore simulates the deployment process, ensuring that all metadata (objects, fields, flows, Apex and dashboards) is properly structured, version-controlled and ready for migration.

7.2 Maintenance, Monitoring and Troubleshooting

Maintenance keeps event, client, vendor and feedback data accurate and up to date. Regular monitoring of flows, Apex jobs and approval processes helps identify failed automations early. Troubleshooting involves:

- Resolving user-access errors and permission conflicts.
- Debugging Apex triggers and flow logic.
- Adjusting validation rules or sharing settings as business needs evolve.

7.3 Project Documentation

This document is the complete record of the EventForce Management System. It captures the business overview and objectives, the phase-wise implementation (requirement, development, UI, testing and security), the data model with ER diagram, the automation components (flows, approval process, validation rule and Apex logic), the security model (profiles, roles, permission sets and sharing rules) and sample screenshots of the Salesforce features. It maps business requirements to Salesforce functionality and serves as a blueprint for future developers and administrators, supporting scalability, training, maintenance and smooth knowledge transfer.

8. Conclusion

The EventForce Management System shows how Salesforce can streamline and automate event-planning operations through customized objects, automation and analytics. Although built within a Developer Org for academic purposes, the project reflects real-world CRM implementation practice by combining a strong data model, automation logic and user experience design.

Key learnings and outcomes:

- Designing a relational data model with Lookup, Master-Detail and junction-object relationships.
- Combining declarative automation (flows, approval processes, validation rules, formula fields) with Apex for advanced logic.
- Building a secure access model with profiles, roles, permission sets, OWD and sharing rules.

- Delivering a unified Lightning app with reports and dashboards for operational insight.

This project marks the culmination of the project lifecycle, with a system that has been tested, documented and prepared for enterprise-level deployment and maintenance.